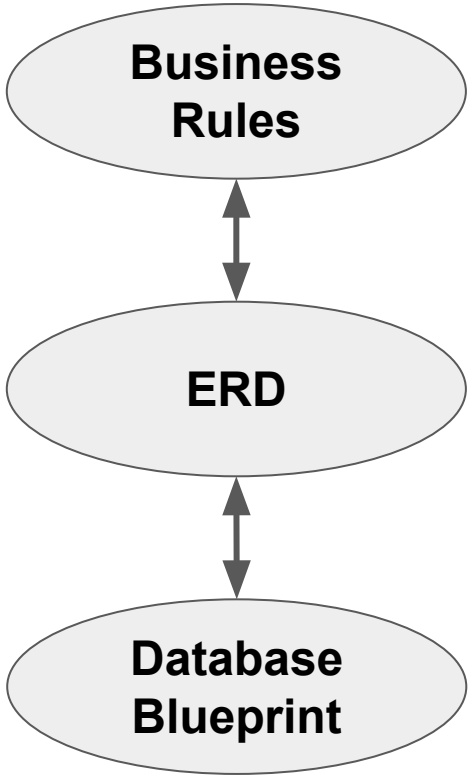
Module 3 – From EER to Relational Schema

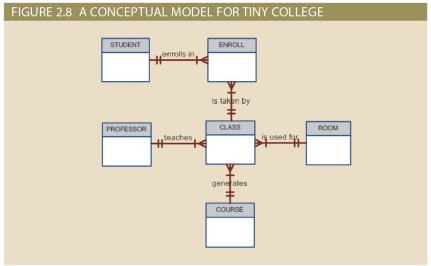
COMP 2714 – Relational Database Systems

Learning Outcome: LO2 – Apply techniques of database design including derivation of relational schema from EER models

Introduction



Business rules **establish entities and the relationships** between them.



?

But, **ERD is not a Database Blueprint**

We **can not implement** a relational database with ERD alone

The Relational Model

- ERD provides a **conceptual model** of data
- To implement a relational database we need an implementation-ready model.
- **Relational model** is the implementation-ready model for relational databases

Implementation-ready model should contain at least 3 components

- Description of **data structures**
- set of enforceable **rules** to guarantee integrity
- **data manipulation methodology** to support real life data transformations

Data Structures

- **Rows** are instances
- **Columns** are attributes
- **Relationships** through common attributes

Rules

- e.g.code can not be **null**
- e.g code should be **unique**

Data Manipulation Methodology

- SQL

AGENT Table

agent_code	agent_last_name	agent_first_name	agent_initial	agent_phone

CUSTOMER Table

customer_code	customer_last_name	customer_first_name	agent_code

Relational Model Basics

Entities → Tables

- Rows & Columns
- Each row represents a single **instance**
- Each column represents an **attribute**
- Each intersection of a row and column is a single data value
- All values in a column have the same **data format**
- Each column has a range of values known as the **attribute domain**
- The order of rows and columns do not matter
- Each table must have an attribute or combination of attributes that uniquely identifies each row → **Primary Key**

Primary Key

- an identifier composed of **one or more attributes** that **uniquely identifies a row.**
- A table often has more than one **candidate** for primary key

To ensure **entity integrity**, the value of all primary keys should be unique and none of the key attributes in the primary key should contain a null

Example - STUDENT table

FIGURE 3.1 STUDENT TABLE ATTRIBUTE VALUES

Table name: STUDENT

Database name: Ch03_TinyCollege

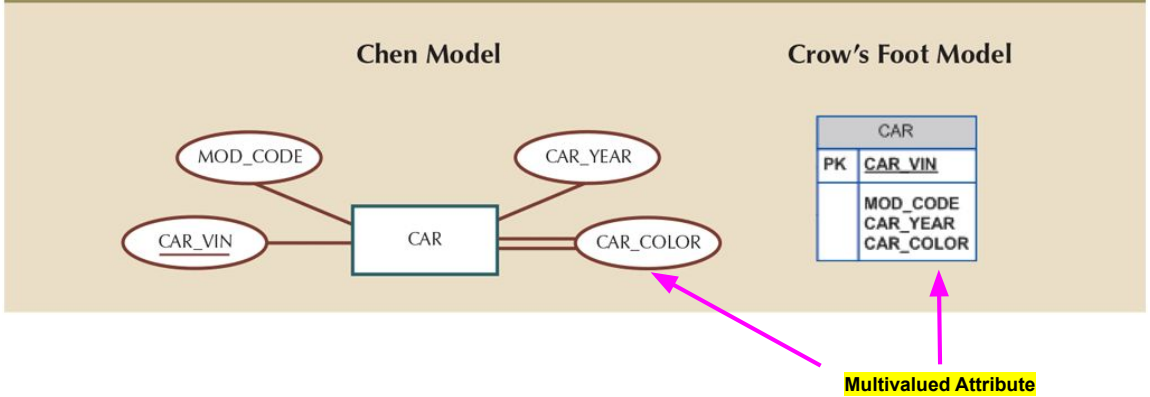
STU_NUM	STU_LNAME	STU_FNAME	STU_INIT	STU_DOB	STU_HRS	STU_CLASS	STU_GPA	STU_TRANSFER	DEPT_CODE	STU_PHONE	PROF_NUM
321452	Bowser	William	C	12-Feb-1985	42	So	2.84	No	BIOL	2134	205
324257	Smithson	Anne	K	15-Nov-1991	81	Jr	3.27	Yes	CIS	2256	222
324258	Brewer	Juliette		23-Aug-1979	36	So	2.26	Yes	ACCT	2256	228
324269	Oblonski	Walter	H	16-Sep-1986	66	Jr	3.09	No	CIS	2114	222
324273	Smith	John	D	30-Dec-1968	102	Sr	2.11	Yes	ENGL	2231	199
324274	Katinga	Raphael	P	21-Oct-1989	114	Sr	3.15	No	ACCT	2267	228
324291	Robertson	Gerald	T	08-Apr-1983	120	Sr	3.87	No	EDU	2267	311
324299	Smith	John	B	30-Nov-1996	15	Fr	2.92	No	ACCT	2315	230

STU_NUM = Student number
STU_LNAME = Student last name
STU_FNAME = Student first name
STU_INIT = Student middle initial
STU_DOB = Student date of birth
STU_HRS = Credit hours earned
STU_CLASS = Student classification
STU_GPA = Grade point average
STU_TRANSFER = Student transferred from another institution
DEPT_CODE = Department code
STU_PHONE = 4-digit campus phone extension
PROF_NUM = Number of the professor who is the student's advisor

Modeling Multivalued Attributes

- in **ER modeling**, attributes can be **multi-valued**
- In the **relational model**, **multi-valued attributes are not allowed**
- They must be **transformed** before implementation
    - (Details → Module 3: Mapping ER → Relational Schema)
- For now: just **recognize them in ERDs** and know they will require special handling

FIGURE 4.3 A MULTIVALUED ATTRIBUTE IN AN ENTITY



How should we implement multivalued attributes?

Implementing Multivalued Attributes

Cannot be directly implemented into an RDBMS

Two ways to implement:

1. Create several attributes

   Only use if all entities have the same number of values for the multivalued attribute and no instance ever has more → otherwise would lead to nulls and is not an acceptable solution

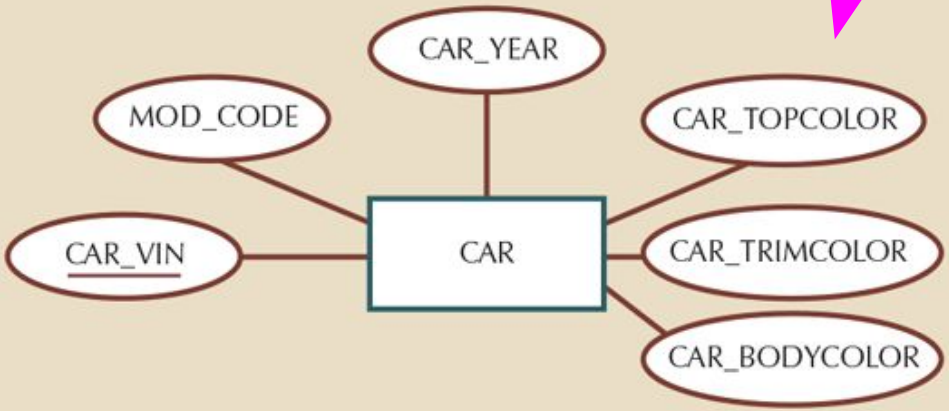
2. New entity to keep attribute's components

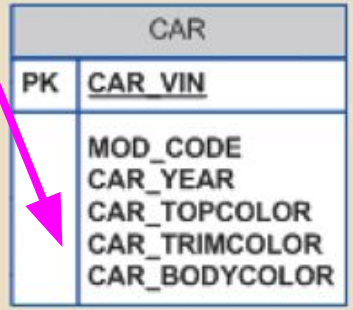
   Preferred solution → flexible & expandable

Implementing Multivalued Attributes

First Solution: Multiple Attributes

FIGURE 4.4 SPLITTING THE MULTIVALUED ATTRIBUTE INTO NEW ATTRIBUTES

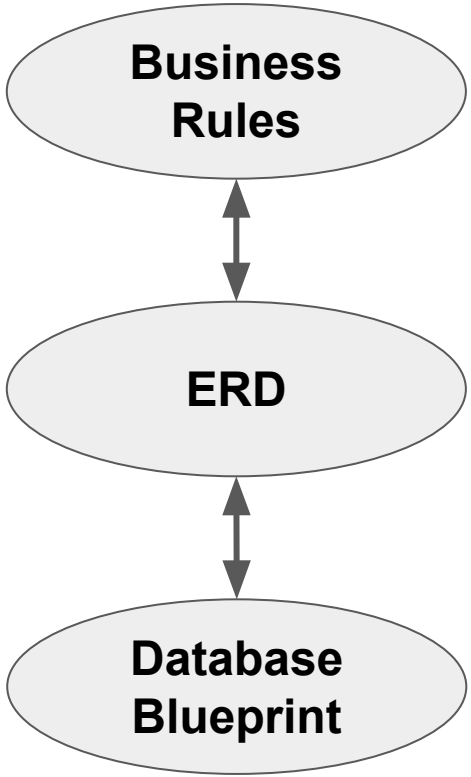




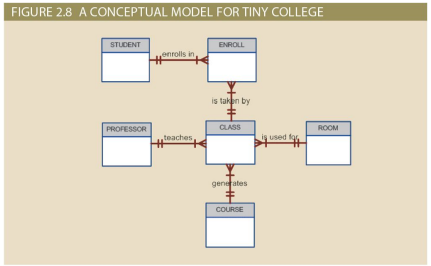
Implementing Multivalued Attributes

Second Solution: New Entity in 1:M relationship

FIGURE 4.5 A NEW ENTITY SET COMPOSED OF A MULTIVALUED ATTRIBUTE'S COMPONENTS



Business rules **establish entities and the relationships** between them.



Entities & Attributes → Tables

How do we implement the **relationships?**

Mapping Relationships

Relationships

One to Many

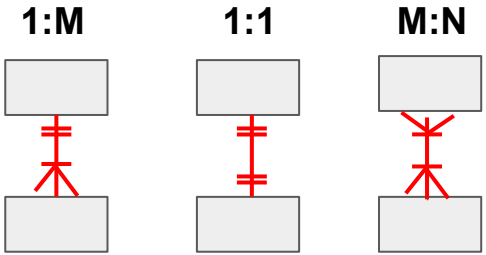
- ideal
- should be the norm in an relational database design

One to One

- should be rare

Many to Many

- cannot be implemented —> should be broken down into two 1:M relationships



1:M Relationship

- implemented by putting the PK of the “1” side in the table of the “Many” side as a **foreign key**

- **foreign key:** the primary key of one table placed in another table to create a common attribute

Referential Integrity

- every **reference to an entity instance by another entity instance** is valid.
- in other words, making sure that what we are linking to **actually exists**.
- the FK should be **either null or a valid value**

1:M Relationship

FIGURE 3.17 THE 1:M RELATIONSHIP BETWEEN PAINTER AND PAINTING

FIGURE 3.18 THE IMPLEMENTED 1:M RELATIONSHIP BETWEEN PAINTER AND PAINTING

Table name: PAINTER
Primary key: PAINTER_NUM
Foreign key: none

Database name: Ch03_Museum

PAINTER_NUM	PAINTER_LNAME	PAINTER_FNAME	PAINTER_INITIAL
123	Ross	Georgette	P
126	Itero	Julio	G

Table name: PAINTING
Primary key: PAINTING_NUM
Foreign key: PAINTER_NUM

PAINTING_NUM	PAINTING_TITLE	PAINTER_NUM
1338	Dawn Thunder	123
1339	Vanilla Roses To Nowhere	123
1340	Tired Flounders	126
1341	Hasty Exit	123
1342	Plastic Paradise	126

1:M Relationship

FIGURE 3.19 THE 1:M RELATIONSHIP BETWEEN COURSE AND CLASS

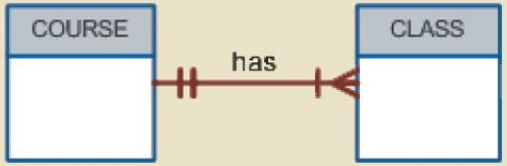


FIGURE 3.20 THE IMPLEMENTED 1:M RELATIONSHIP BETWEEN COURSE AND CLASS

Table name: COURSE
Primary key: CRS_CODE
Foreign key: none

Database name: Ch03_TinyCollege

CRS_CODE	DEPT_CODE	CRS_DESCRIPTION	CRS_CREDIT
ACCT-211	ACCT	Accounting I	3
ACCT-212	ACCT	Accounting II	3
CIS-220	CIS	Intro. to Microcomputing	3
CIS-420	CIS	Database Design and Implementation	4
QM-261	CIS	Intro. to Statistics	3
QM-362	CIS	Statistical Applications	4

Table name: CLASS
Primary key: CLASS_CODE
Foreign key: CRS_CODE

CLASS_CODE	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10012	ACCT-211	1	MWF 8:00-8:50 a.m.	BUS311	105
10013	ACCT-211	2	MWF 9:00-9:50 a.m.	BUS200	105
10014	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10015	ACCT-212	1	MWF 10:00-10:50 a.m.	BUS311	301
10016	ACCT-212	2	Th 6:00-8:40 p.m.	BUS252	301
10017	CIS-220	1	MWF 9:00-9:50 a.m.	KLR209	228
10018	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10019	CIS-220	3	MWF 10:00-10:50 a.m.	KLR209	228
10020	CIS-420	1	W 6:00-8:40 p.m.	KLR209	162
10021	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114
10022	QM-261	2	TTh 1:00-2:15 p.m.	KLR200	114
10023	QM-362	1	MWF 11:00-11:50 a.m.	KLR200	162
10024	QM-362	2	TTh 2:30-3:45 p.m.	KLR200	162

Implementing 1:1 Relationships

- Placing a FK in both entities is not recommended as it duplicates work and might also interfere with other relationships between the two entity.
  - e.g. consider EMPLOYEE chairs DEPARTMENT (1:1) and EMPLOYEE works in a DEPARTMENT (1:M)
- So we have to place the **FK in one of the tables**. But which one?

1:1 Relationship

FIGURE 3.22 THE IMPLEMENTED 1:1 RELATIONSHIP BETWEEN PROFESSOR AND DEPARTMENT

Table name: PROFESSOR
Primary key: EMP_NUM
Foreign key: DEPT_CODE

Database name: Ch03_TinyCollege

EMP_NUM	DEPT_CODE	PROF_OFFICE	PROF_EXTENSION	PROF_HIGH_DEGREE
103	HIST	DRE 156	6783	Ph.D.
104	ENG	DRE 102	5561	MA
105	ACCT	KLR 229D	8665	Ph.D.
106	MKT/MGT	KLR 126	3899	Ph.D.
110	BIOL	AAK 160	3412	Ph.D.
114	ACCT	KLR 211	4436	Ph.D.
155	MATH	AAK 201	4440	Ph.D.
160	ENG	DRE 102	2248	Ph.D.
162	CIS	KLR 203E	2359	Ph.D.
191	MKT/MGT	KLR 409B	4016	DBA
195	PSYCH	AAK 297	3550	Ph.D.
209	CIS	KLR 333	3421	Ph.D.
228	CIS	KLR 300	3000	Ph.D.
297	MATH	AAK 194	1145	Ph.D.
299	ECON/FIN	KLR 284	2851	Ph.D.
301	ACCT	KLR 244	4683	Ph.D.
335	ENG	DRE 208	2000	Ph.D.
342	SOC	BBG 208	5514	Ph.D.
387	BIOL	AAK 230	8665	Ph.D.
401	HIST	DRE 156	6783	MA
425	ECON/FIN	KLR 284	2851	MBA
435	ART	BBG 185	2278	Ph.D.

The 1:M DEPARTMENT employs PROFESSOR relationship is implemented through the placement of the DEPT_CODE foreign key in the PROFESSOR table.

The 1:1 PROFESSOR chairs DEPARTMENT relationship is implemented through the placement of the EMP_NUM foreign key in the DEPARTMENT table.

Table name: DEPARTMENT
Primary key: DEPT_CODE
Foreign key: EMP_NUM

DEPT_CODE	DEPT_NAME	SCHOOL_CODE	EMP_NUM	DEPT_ADDRESS	DEPT_EXTENSION
ACCT	Accounting	BUS	114	KLR 211, Box 52	3119
ART	Fine Arts	A&SCI	435	BBG 185, Box 128	2278
BIOL	Biology	A&SCI	387	AAK 230, Box 415	4117
CIS	Computer Info. Systems	BUS	209	KLR 333, Box 56	3245
ECON/FIN	Economics/Finance	BUS	299	KLR 284, Box 63	3126
ENG	English	A&SCI	160	DRE 102, Box 223	1004
HIST	History	A&SCI	103	DRE 156, Box 284	1867
MATH	Mathematics	A&SCI	297	AAK 194, Box 422	4234
MKT/MGT	Marketing/Management	BUS	106	KLR 126, Box 55	3342
PSYCH	Psychology	A&SCI	195	AAK 297, Box 438	4110
SOC	Sociology	A&SCI	342	BBG 208, Box 132	2008

FIGURE 3.21 THE 1:1 RELATIONSHIP BETWEEN PROFESSOR AND DEPARTMENT

- We could potentially put the PK of DEPARTMENT into the PROFESSOR table. But that would be less ideal as not all professors are department heads (leads to lots of nulls). But each department has a head.

Null Values

- nulls should be **avoided as much as possible**
- abundance of nulls is **a sign of poor design**

TABLE 5.5

SELECTION OF FOREIGN KEY IN A 1:1 RELATIONSHIP

CASE	ER RELATIONSHIP CONSTRAINTS	ACTION
I	One side is mandatory and the other side	Place the PK of the entity on the mandatory side in the entity on the optional side as a FK, and make the FK mandatory.
II	Both sides are optional.	Select the FK that causes the fewest nulls, or place the FK in the entity in which the (relationship) role is played.
III	Both sides are mandatory.	See Case II, or consider revising your model to ensure that the two entities do not belong together in a single entity.

Implementing 1:1 Relationships

Example: One side is mandatory and the other side is optional.

FIGURE 5.8 THE 1:1 RELATIONSHIP BETWEEN DEPARTMENT AND EMPLOYEE

A One-to-One (1:1) Relationship:
An EMPLOYEE manages zero or one DEPARTMENT;
each DEPARTMENT is managed by one EMPLOYEE.

M:N Relationship

- **Relationships** are implemented through common attributes.
- Which common attributes should we defined among the two tables?

FIGURE 3.23 THE ERM'S M:N RELATIONSHIP BETWEEN STUDENT AND CLASS

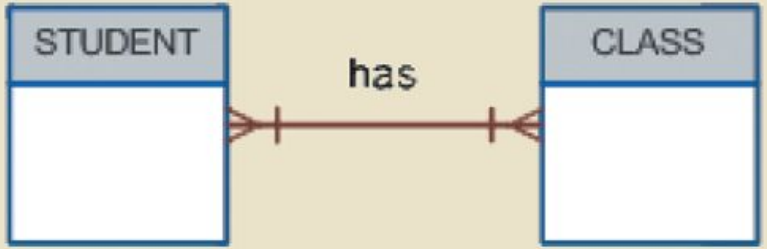


FIGURE 3.24 THE WRONG IMPLEMENTATION OF THE M:N RELATIONSHIP BETWEEN STUDENT AND CLASS

Table name: STUDENT
Primary key: STU_NUM
Foreign key: none

Database name: Ch03_CollegeTry

STU_NUM	STU_LNAME	CLASS_CODE
321452	Bowser	10014
321452	Bowser	10018
321452	Bowser	10021
324257	Smithson	10014
324257	Smithson	10018
324257	Smithson	10021

Table name: CLASS
Primary key: CLASS_CODE
Foreign key: STU_NUM

CLASS_CODE	STU_NUM	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10014	321452	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10014	324257	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10018	321452	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10018	324257	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10021	321452	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114
10021	324257	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114

M:N Relationship - Design 1

- the above implementation is **not good**
- **Many redundancies** —> leads to Anomalies

FIGURE 3.24 THE WRONG IMPLEMENTATION OF THE M:N RELATIONSHIP BETWEEN STUDENT AND CLASS

Table name: STUDENT
Primary key: STU_NUM
Foreign key: none

Database name: Ch03_CollegeTry

STU_NUM	STU_LNAME	CLASS_CODE
321452	Bowser	10014
321452	Bowser	10018
321452	Bowser	10021
324257	Smithson	10014
324257	Smithson	10018
324257	Smithson	10021

Table name: CLASS
Primary key: CLASS_CODE
Foreign key: STU_NUM

CLASS_CODE	STU_NUM	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10014	321452	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10014	324257	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10018	321452	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10018	324257	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10021	321452	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114
10021	324257	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114

M:N Relationship - Design 1

- the above implementation is **not good**
- **Many redundancies** —> leads to Anomalies
- the relational **operations become very complex** —> leads to system efficiency errors and output errors

Changing the time of class 10018

- one row for each student
  - **as much as the number of students registered in the class, we have to do update operations**

FIGURE 3.25 CONVERTING THE M:N RELATIONSHIP INTO TWO 1:M RELATIONSHIPS

Database name: Ch03_CollegeTry2

Table name: STUDENT
Primary key: STU_NUM
Foreign key: none

STU_NUM	STU_LNAME
321452	Bowser
324257	Smithson

Table name: ENROLL
Primary key: CLASS_CODE + STU_NUM
Foreign key: CLASS_CODE, STU_NUM

CLASS_CODE	STU_NUM	ENROLL_GRADE
10014	321452	C
10014	324257	B
10018	321452	A
10018	324257	B
10021	321452	C
10021	324257	C

Table name: CLASS
Primary key: CLASS_CODE
Foreign key: CRS_CODE

CLASS_CODE	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10014	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10018	CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
10021	QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114

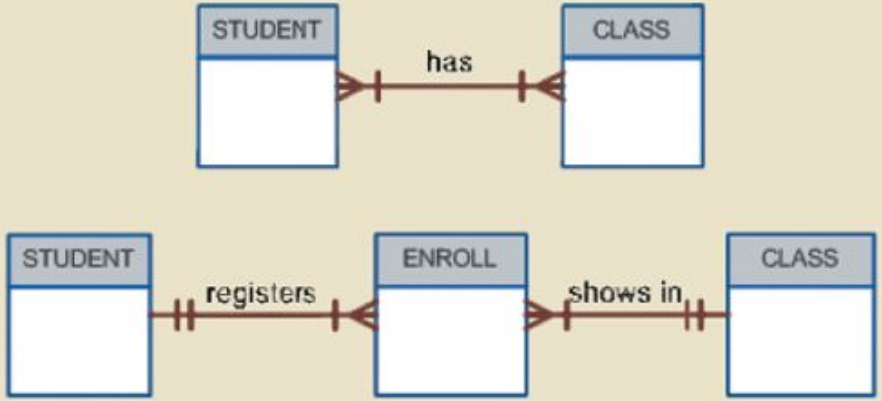
the linking table can contain **any relevant attribute**

controlled redundancy in the ENROLL table **will not lead to anomalies if** the referential integrity is enforced

M:N Relationship - Design 2

- The right way
- create a composite entity
- also called **bridge entity** or associative entity
- includes the PK of the two tables that should be related
- **PK of the composite entity** can be a composite key (combination of the two PKs) or the designer can introduce a new PK

FIGURE 3.26 CHANGING THE M:N RELATIONSHIPS TO TWO 1:M RELATIONSHIPS



Implementing Recursive Relationships (1:1)

- James is married to Louise, Louise is married to James (shared last name)
- Anne is married to Anton, Anton is married to Anne (different last names)

FIGURE 4.18 THE 1:1 RECURSIVE RELATIONSHIP "EMPLOYEE IS MARRIED TO EMPLOYEE"

Database name: Ch04_PartCo

Table name: EMPLOYEE_V1

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_SPOUSE
345	Ramirez	James	347
346	Jones	Anne	349
347	Ramirez	Louise	345
348	Delaney	Robert	
349	Shapiro	Anton	346

Implementing Recursive Relationships (1:M)

FIGURE 4.22 IMPLEMENTATION OF THE 1:M RECURSIVE RELATIONSHIP "EMPLOYEE MANAGES EMPLOYEE"

Database name: Ch04_PartCo

Table name: EMPLOYEE_V2

EMP_CODE	EMP_LNAME	EMP_MANAGER
101	Waddell	102
102	Orincona	
103	Jones	102
104	Reballoh	102
105	Robertson	102
106	Deltona	102

Implementing Recursive Relationships (M:N)

FIGURE 4.21 IMPLEMENTATION OF THE M:N RECURSIVE RELATIONSHIP "COURSE REQUIRES COURSE"

Table name: COURSE **Database name: Ch04_TinyCollege**

CRS_CODE	DEPT_CODE	CRS_DESCRIPTION	CRS_CREDIT
ACCT-211	ACCT	Accounting I	3
ACCT-212	ACCT	Accounting II	3
CIS-220	CIS	Intro. to Microcomputing	3
CIS-420	CIS	Database Design and Implementation	4
MATH-243	MATH	Mathematics for Managers	3
QM-261	CIS	Intro. to Statistics	3
QM-362	CIS	Statistical Applications	4

Table name: PREREQ

CRS_CODE	PRE_TAKE
CIS-420	CIS-220
QM-261	MATH-243
QM-362	MATH-243
QM-362	QM-261

Implementing Recursive Relationships (M:N)

FIGURE 4.20 THE IMPLEMENTATION OF THE M:N RECURSIVE RELATIONSHIP "PART CONTAINS PART"

Table name: COMPONENT

Database name: Ch04_PartCo

COMP_CODE	PART_CODE	COMP_PARTS_NEEDED
C-130	AA21-6	4
C-130	AB-121	2
C-130	E129	1
C-131A2	E129	1
C-130	X10	4
C-131A2	X10	1
C-130	X34AW	2
C-131A2	X34AW	2

Table name: PART

PART_CODE	PART_DESCRIPTION	PART_IN_STOCK
AA21-6	2.5 cm. washer, 1.0 mm. rim	432
AB-121	Cotter pin, copper	1034
C-130	Rotor assembly	36
E129	2.5 cm. steel shank	128
X10	10.25 cm. rotor blade	345
X34AW	2.5 cm. hex nut	879

If PART consists of many PARTs and can be part of many, we need **two tables**

Part tracking is very common & important practice which is required by law for some industries, e.g. Aviation

Existence Dependence

- Some entities, we should keep in the database, only if they are related to another entity in our system
- Example: "**EMPLOYEE Claims DEPENDENT**"
- If the employee is removed from the database, we clearly should remove his dependents from the database as well.

How do we implement existence dependence?

Implementing Existence Dependence

"An entity is existence-dependent if it has a **mandatory foreign key** -- that is, a foreign key attribute that **cannot be null**"

FIGURE 4.9 A STRONG (IDENTIFYING) RELATIONSHIP BETWEEN COURSE AND CLASS

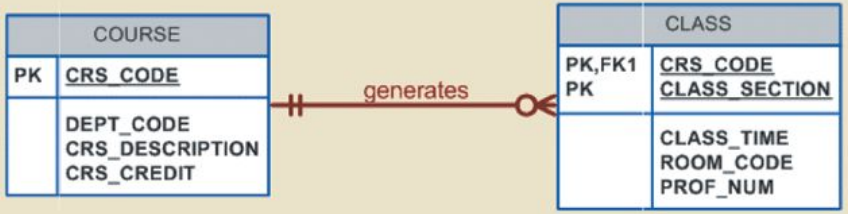


Table name: COURSE

Database name: Ch04_TinyCollege_Alt

CRS_CODE	DEPT_CODE	CRS_DESCRIPTION	CRS_CREDIT
ACCT-211	ACCT	Accounting I	3
ACCT-212	ACCT	Accounting II	3
CIS-220	CIS	Intro. to Microcomputing	3
CIS-420	CIS	Database Design and Implementation	4
MATH-243	MATH	Mathematics for Managers	3
QM-261	CIS	Intro. to Statistics	3
QM-362	CIS	Statistical Applications	4

Table name: CLASS

CRS_CODE	CLASS_SECTION	CLASS_TIME	ROOM_CODE	PROF_NUM
ACCT-211	1	MWF 8:00-8:50 a.m.	BUS311	105
ACCT-211	2	MWF 9:00-9:50 a.m.	BUS200	105
ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
ACCT-212	1	MWF 10:00-10:50 a.m.	BUS311	301
ACCT-212	2	Th 6:00-8:40 p.m.	BUS252	301
CIS-220	1	MWF 9:00-9:50 a.m.	KLR209	228
CIS-220	2	MWF 9:00-9:50 a.m.	KLR211	114
CIS-220	3	MWF 10:00-10:50 a.m.	KLR209	228
CIS-420	1	W 6:00-8:40 p.m.	KLR209	162
MATH-243	1	Th 6:00-8:40 p.m.	DRE155	325
QM-261	1	MWF 8:00-8:50 a.m.	KLR200	114
QM-261	2	TTh 1:00-2:15 p.m.	KLR200	114
QM-362	1	MWF 11:00-11:50 a.m.	KLR200	162
QM-362	2	TTh 2:30-3:45 p.m.	KLR200	162

Implementing Strong Relationships

"**The order** in which tables are created and loaded is very important"

Usually the parent's table should be created and loaded before the child entities.

Why ?

Implementing Specialization/Generalization Hierarchies

Goal: Map an EER supertype–subtype structure into relational schema.

Three common strategies:

1. Single Table (All in One Relation)
2. Multiple Tables (One per Subtype, No Supertype Table)
3. Supertype + Subtype Tables (Recommended)

1. Single Table (All in One Relation)

- One relation for the supertype + all subtype attributes.
- Subtype-specific columns left NULL when not applicable.

Example:

```sql
EMPLOYEE(emp_id, name, hire_date, license_no, acct_cert, mech_grade)
```

Pros:

- Simple design.
- Easy to query (no joins).

Cons:

- Many NULLs.
- Poor normalization.
- Can become unwieldy if many subtypes.

2. Multiple Tables (One per Subtype, No Supertype Table)

- Each subtype is a separate relation.
- Common attributes repeated in every table.

Example:

```sql
PILOT(emp_id, name, hire_date, license_no)
ACCOUNTANT(emp_id, name, hire_date, acct_cert)
MECHANIC(emp_id, name, hire_date, mech_grade)
```

Pros:

- No NULLs.
- Tables mirror real-world categories.

Cons:

- Redundancy in common attributes.
- Updates to common data require multiple changes.

3. Supertype + Subtype Tables (Recommended)

- One relation for the supertype with common attributes.
- One relation per subtype with PK = FK to supertype.

Example:

```sql
EMPLOYEE(emp_id, name, hire_date)
PILOT(emp_id, license_no)
ACCOUNTANT(emp_id, acct_cert)
MECHANIC(emp_id, mech_grade)
```

Pros:

- No redundancy.
- No NULLs.
- Clean separation, easy enforcement of constraints.

Cons:

- Requires joins for queries across all subtypes.

Implementing Specialization/Generalization Hierarchies

Goal: Map an EER supertype–subtype structure into relational schema.

Three common strategies:

1. Single Table (All in One Relation)
2. Multiple Tables (One per Subtype, No Supertype Table)
3. Supertype + Subtype Tables (Recommended)

Key Notes

- Works with **disjoint vs. overlapping** and **total vs. partial** constraints.
- Textbook best practice is usually **Strategy 3** (supertype + subtype tables).
- Strategy choice may depend on performance, data sparsity, or business rules.

Keys & Constraints

Selecting (Good) Primary Keys

Entity Integrity & Primary Key

- The **most important characteristic** of an entity is its Primary Key
  - Remember that a primary key could be either **a single attribute or a combination of multiple attributes**
- It is the PK that uniquely identifies every instance of an entity. (Every row of the table)
  - If you can't uniquely identify the individual instances, then the entity doesn't show **entity integrity.**
- The **efficiency and effectiveness of your DB** is directly affected by your choice of primary keys.

Natural Keys & Primary Keys

- In the real world, sometimes there are **natural identifiers**.
  - e.g. A car plate number
- Natural keys are **familiar** to end users
- If an entity has a natural identifier, use that as the primary key of the entity when you are modelling.
- Most natural keys make **acceptable primary keys** but make sure that the natural key is also a good key.

What counts as a good primary key?!

- How many attributes is too many attributes to have in a PK?
- How long can a PK be?
- Why do we choose EMP_NUM as PK and not a combination of EMP_LNAME, EMP_FNAME and EMP_DOB?
- To answer questions, we need to understand a few things:
  1. We have to distinguish between **identification vs. description**.
  2. We have to consider the effect that the choice of PKs have on the **implementation of relationships**.

PKs are not to describe the entity. They are to be used to **uniquely identify** the entity, so that the value of other (descriptive) attributes can be quickly found.

Desirable Primary Keys

- Unique values
- Non-intelligent
- No change over time
- Preferably single-attribute
- Preferably numeric
- Security-compliant

TABLE 5.3

DESIRABLE PRIMARY KEY CHARACTERISTICS

PK CHARACTERISTIC	RATIONALE
Unique values	The PK must uniquely identify each entity instance. A primary key must be able to guarantee unique values. It cannot contain nulls.
Nonintelligent	The PK should not have embedded semantic meaning other than to uniquely identify each entity instance. An attribute with embedded semantic meaning is probably better used as a descriptive characteristic of the entity than as an identifier. For example, a student ID of 650973 would be preferred over Smith, Martha L. as a primary key identifier.
No change over time	If an attribute has semantic meaning, it might be subject to updates, which is why names do not make good primary keys. If Vickie Smith is the primary key, what happens if she changes her name when she gets married? If a primary key is subject to change, the foreign key values must be updated, thus adding to the database work load. Furthermore, changing a primary key value means that you are basically changing the identity of an entity. In short, the PK should be permanent and unchangeable.
Preferably single-attribute	A primary key should have the minimum number of attributes possible (irreducible). Single-attribute primary keys are desirable but not required. Single-attribute primary keys simplify the implementation of foreign keys. Having multiple-attribute primary keys can cause primary keys of related entities to grow through the possible addition of many attributes, thus adding to the database workload and making (application) coding more cumbersome.
Preferably numeric	Unique values can be better managed when they are numeric, because the database can use internal routines to implement a counter-style attribute that automatically increments values with the addition of each new row. In fact, most database systems include the ability to use special constructs, such as Autonumber in Microsoft Access, sequence in Oracle, or uniqueidentifier in MS SQL Server to support self-incrementing primary key attributes.
Security-compliant	The selected primary key must not be composed of any attribute(s) that might be considered a security risk or violation. For example, using a Social Security number as a PK in an EMPLOYEE table is not a good idea.

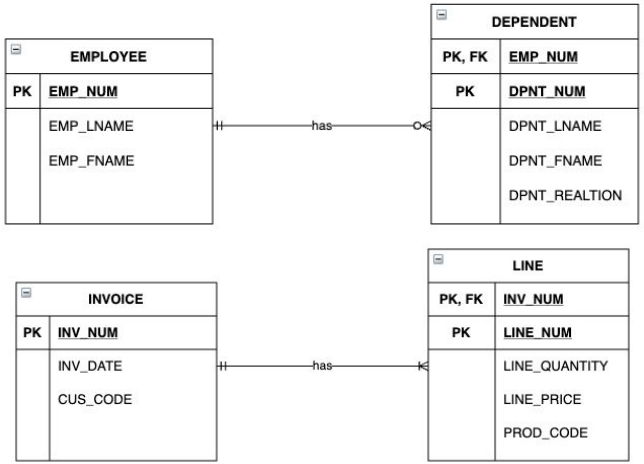
When to use composite primary keys?

- When we say that we prefer PKs to be single-valued, that doesn't mean composite primary keys are not allowed.
- Composite keys are very useful as:
  - identifiers in **composite entities.** e.g. in ENROLL: a student cannot enrol more than once in the same class.
  - identifiers of **weak entities**.

Composite PKs for Weak Entities

Remember that weak entities are usually used to represent one of the two situations below:

- A real-world object that can only be in the database because of **existence-dependence** on another entity. Even though the two entities might be independent in the real-world.e.g. EMPLOYEE & DEPENDENT
- When a real-world object is **represented by multiple entities** in our models. e.g. INVOICE & LINE



In both cases, using a composite primary key is a good choice as it ensures the weak entity has a strong identifying relationship to its parent entity.

When to use surrogate primary keys?

- When we do not have a natural key, or the natural key doesn't make an appropriate PK, we can use Surrogate Keys.
- Surrogate Key (Also called **Codes**) are **designed by the DB designer**. To simplify identification of instances (not description).
  - Has **no meaning** in the user environment
  - **DBMS can generate** values (codes) for us. Since there doesn't have to be any meaning.
- Surrogate keys are accepted practice in today's complex data environments.

When to use surrogate primary keys?

- Is there a good natural key?
- Remember that if you choose multiple attributes for PK just because a single attribute natural key cannot be found, then you will **complicate the implementation** of relationships through FKs.

TABLE 5.4

DATA USED TO KEEP TRACK OF EVENTS

DATE	TIME_START	TIME_END	ROOM	EVENT_NAME	PARTY_OF
6/17/2018	11:00 a.m.	2:00 p.m.	Allure	Burton Wedding	60
6/17/2018	11:00 a.m.	2:00 p.m.	Bonanza	Adams Office	12
6/17/2018	3:00 p.m.	5:30 p.m.	Allure	Smith Family	15
6/17/2018	3:30 p.m.	5:30 p.m.	Bonanza	Adams Office	12
6/18/2018	1:00 p.m.	3:00 p.m.	Bonanza	Boy Scouts	33
6/18/2018	11:00 a.m.	2:00 p.m.	Allure	March of Dimes	25
6/18/2018	11:00 a.m.	12:30 p.m.	Bonanza	Smith Family	12

Complicating the implementation (of both database) and applications because of lengthy primary keys is **unnecessary**. Instead use a numeric, single-attribute surrogate key!

IMPORTANT: if you use a surrogate key, you still need to ensure that the **candidate key** of the entity in question *performs properly* through the use of "**unique index**" and "**not null**" constraints.

Constraints in Relational Schema

Constraints = rules that ensure data validity & consistency

Types:

- **Entity integrity** (primary key: unique, non-null)
- **Referential integrity** (foreign keys must match a valid value in parent relation)
- **Attribute constraints** (e.g., NOT NULL, uniqueness, allowed values)

Constraints capture **business rules** beyond just structure

Entity & Referential Integrity

Entity integrity

- Every relation must have a PK
- PK must uniquely identify tuples
- PK attributes cannot be NULL

Referential integrity

- FK values must exist in the referenced relation
- Prevents “orphan” records

Attribute & Business Rule Constraints

Attribute-level

- NOT NULL: value required
- Uniqueness: no duplicates allowed
- Simple checks (e.g., grade $\in$ {A,B,C,D,F})

Business rules (sometimes can be enforced as constraints)

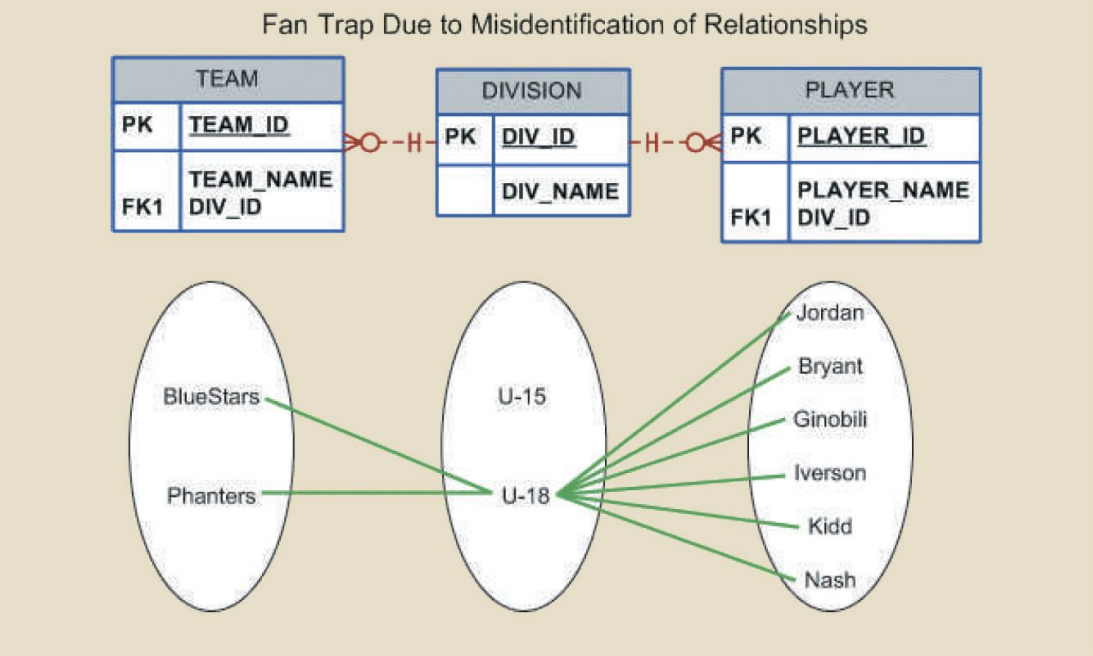
- Example: end_date $\geq$ start_date

In practice: constraints ensure your relational schema is both **valid** and **faithful** to requirements

Special Design Cases

Fan Traps

FIGURE 5.12 INCORRECT ERD WITH FAN TRAP PROBLEM

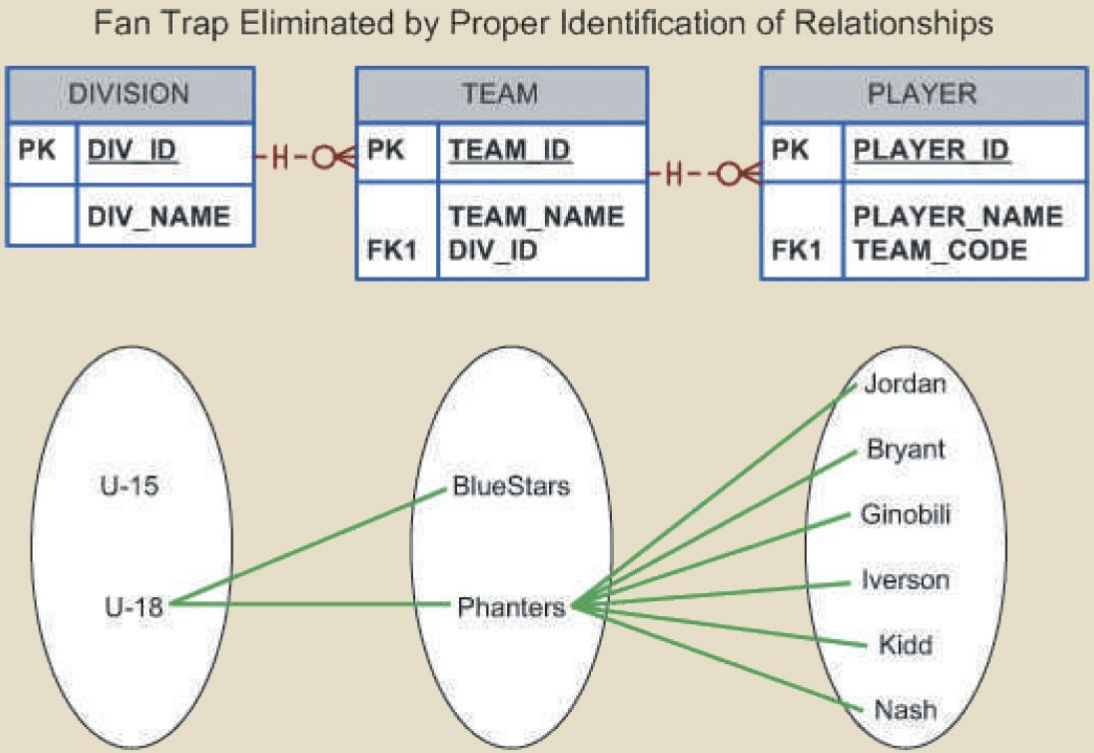


The most common **design trap**

We should eliminate Fan traps as they are usually a sign of **misidentified relationships**.

- E.g. Notice that even though everything is semantically correct, there is no way to find out which player plays in which team!

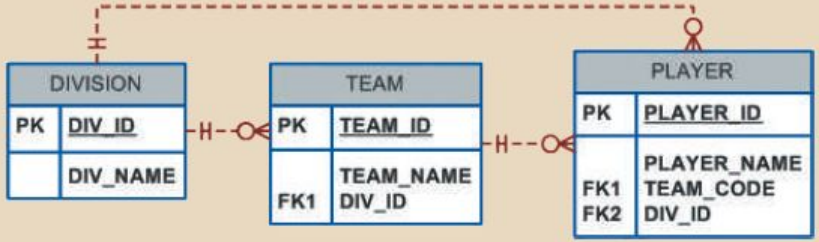
FIGURE 5.13 CORRECTED ERD AFTER REMOVAL OF THE FAN TRAP



Redundant Relationships

- When there are **multiple relationship path** between related entities, we say there is relationship redundancy.
- Redundant relationships are **sometimes used to simplify the design**.
- But **sometimes there is no justification** for the redundant relationships and it can be safely removed.

FIGURE 5.14 A REDUNDANT RELATIONSHIP

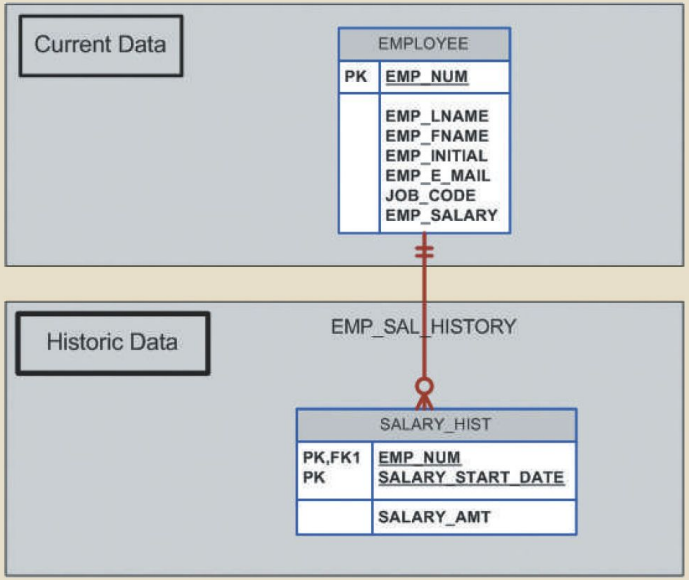


Maintaining History of Time-Variant Data

- **Time-Variant Data**: data whose values change over time and for which you must keep a history of data changes.
- The storage of time-variant data requires **changes in the data model:**
  - Sometimes keeping history means **an attribute becomes multi-valued**.
  - Sometimes, the necessity to maintain historic data **converts a 1:M or 1:1 relationship to a M:N relationship!**

Maintaining History: Multi-Valued Attribute

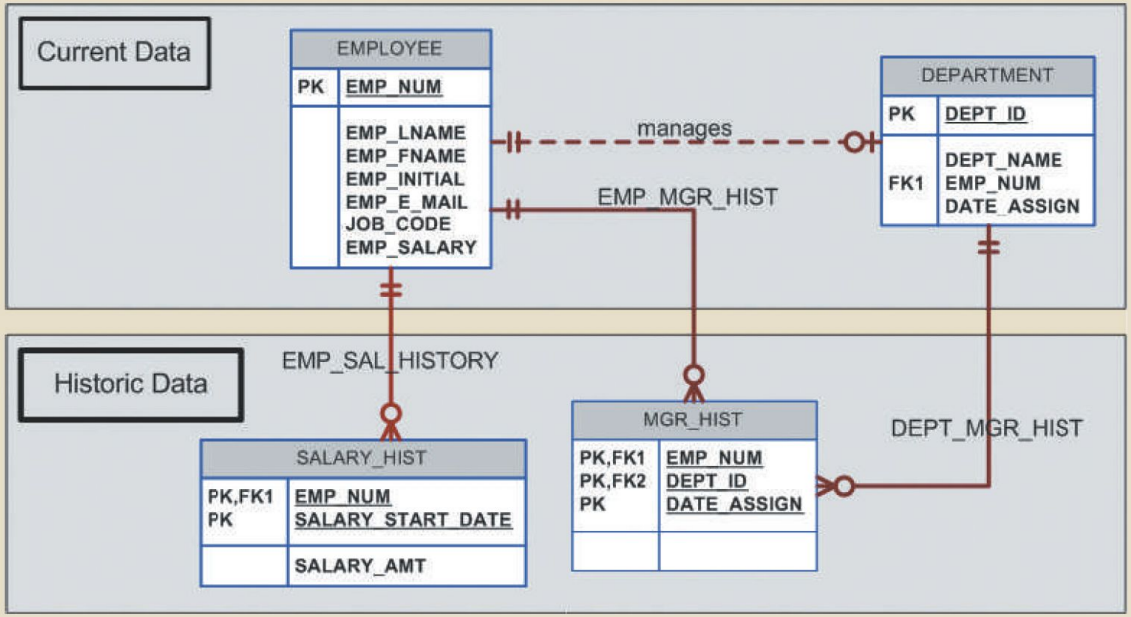
FIGURE 5.9 MAINTAINING SALARY HISTORY



Sometimes keeping history means **an attribute becomes multi-valued**. e.g. Employee Salary

Maintaining History: M:N Relationship

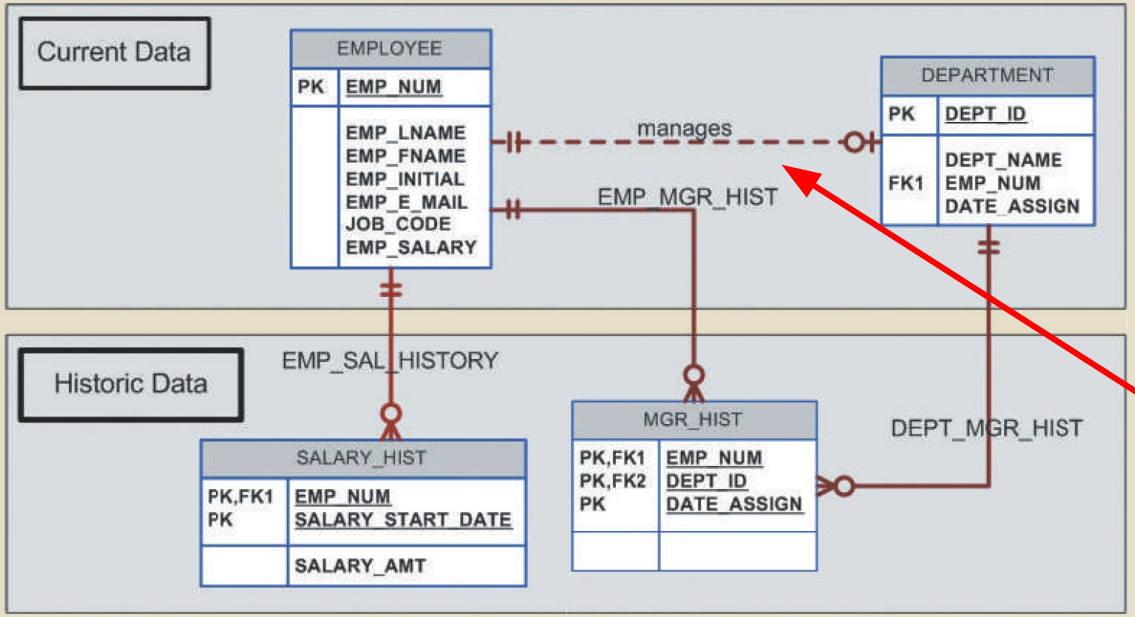
FIGURE 5.10 MAINTAINING MANAGER HISTORY



Sometimes, the necessity to maintain historic data converts a 1:M or 1:1 relationship to a **M:N relationship**! e.g. EMPLOYEE manages DEPARTMENT.

Maintaining History: M:N Relationship

FIGURE 5.10 MAINTAINING MANAGER HISTORY



Sometimes, the necessity to maintain historic data converts a 1:M or 1:1 relationship to a **M:N relationship**! e.g. EMPLOYEE manages DEPARTMENT.

Notice that the "manages" relationship is still kept even though the current manager of a department can be queried from the MGR_HIST table. Why?

Takeaways & Next Steps

By the end of this module, you should be able to:

- Map **entities and attributes** into relational tables and columns
- Implement **primary keys** (natural, composite, surrogate) and enforce **entity integrity**
- Implement **foreign keys** and enforce **referential integrity**
- Map **relationships** into relational schemas:
  - 1:M (FK on “many” side)
  - 1:1 (design options for FK placement)
  - M:N (bridge/composite entities)
  - Recursive relationships
- Handle **existence dependence** and **strong relationships** in schema design
- Recognize and design for **special cases**: multi-valued attributes, time-variant data, fan traps, redundant relationships

Next Steps → Module 4

- Refine relational schemas through **functional dependencies**
- Apply **normalization** (1NF → BCNF) to ensure consistency and reduce redundancy